

Αλγόριθμοι

3η Σειρά Προγραμματιστικών Ασκήσεων

Παναγή Αντρέας
Α.Μ. : 7115 1424 00008

Π.Μ.Σ. - ΑΛΜΑ

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα *

Φεβρουάριος 2024



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

* Στοιχειοθέτηση με χρήση $\text{\LaTeX}$

Περιεχόμενα

Άσκηση 1: Συντόμευση Διαδρομής	1
Εκφώνηση	1
Λύση	1
Προσέγγιση Λύσης:	1
Υλοποίηση Αλγορίθμων	1
Σημειώσεις:	3

Άσκηση 1: Συντόμευση Διαδρομής

Εκφώνηση

Είμαστε στην αγαπημένη μας παραλία, όπου έχουμε πάει για μια χειμερινή βουτιά, και πρέπει να φτάσουμε σπίτι μας, για μια πολύ σημαντική τηλεδιάσκεψη, σε B ώρες ακριβώς!

Γνωρίζουμε το οδικό δίκτυο της χώρας, το οποίο αποτελείται από N πόλεις (αριθμημένες από 1 μέχρι N) και M δρόμους μονής κατεύθυνσης. Κάθε δρόμος $e = (u, v)$ μας επιτρέπει να ταξιδέψουμε από την πόλη u , στη μία άκρη του, στην πόλη v , στην άλλη άκρη, σε $\ell(e)$ ώρες. Η αγαπημένη μας παραλία βρίσκεται στην πόλη s και το σπίτι μας στην πόλη t . Υπό κανονικές συνθήκες, φαίνεται αδύνατον να διανύσουμε την απόσταση από την πόλη s στην πόλη t σε B μόλις ώρες. Ευτυχώς, όμως, το υπερσύγχρονο αυτοκίνητό μας έχει τη δυνατότητα να κινηθεί αστραπιαία κατά μήκος ενός δρόμου. Όταν συμβαίνει αυτό σε έναν δρόμο $e = (u, v)$, διανύουμε την απόσταση από την πόλη u στην πόλη v σε ελάχιστα λεπτά, μηδενίζοντας πρακτικά το $\ell(e)$. Δυστυχώς αυτή η δυνατότητα μπορεί να χρησιμοποιηθεί λίγες φορές σε όλη τη διάρκεια ζωής του αυτοκινήτου μας.

Θέλουμε λοιπόν να υπολογίσουμε το ελάχιστο πλήθος δρόμων, όπου αν κινηθούμε αστραπιαία, μηδενίζοντας πρακτικά το χρόνο που χρειάζεται για να τους διασχίσουμε, θα καταφέρουμε να φτάσουμε από την αγαπημένη μας παραλία, στην πόλη s , στο σπίτι μας, στην πόλη t , σε B ώρες το πολύ, ώστε να είμαστε στην ώρα μας για την τηλεδιάσκεψη.

Λύση

Προσέγγιση Λύσης:

Θα χρησιμοποιήσουμε το πρόβλημα απόφασης (decision problem) για να λύσουμε το πρόβλημα βελτιστοποίησης (optimization problem).

Το προφανές ανώτατο όριο διαγραφής ακμών είναι το $M = m(G) = |E|$ και αντίστοιχα το κατώτατο όριο διαγραφής ακμών είναι 1 ακμή.

Θα εκμεταλλευτούμε την μονοτονία των απαντήσεων ώστε να βρούμε το ελάχιστο πλήθος. Με άλλα λόγια παρατηρούμε ότι οι απαντήσεις του decision problem ρωτώντας για πλήθος διαγραφών: $0, 1, \dots, M$ θα είναι OXI,...,OXI,NAI,...,NAI.

Επομένως, εκμεταλλευόμενοι την δομή των απαντήσεων, θα χρησιμοποιήσουμε δυαδική αναζήτηση ώστε να βρούμε το πρώτο (ελάχιστο) πλήθος διαγραφών ακμών ώστε να πάρουμε απάντηση NAI.

Υλοποίηση Αλγορίθμων

Αλγόριθμος 1: Optimazation_Problem($G = (V, E, l), s, t, B$)

Είσοδος: Κατευθυνόμενο γράφημα $G = (V, E, l)$ με συνάρτηση βαρών $l : E \rightarrow \mathbb{N}$, $s, t \in V(G)$ και μέγιστο κόστος $B \in \mathbb{N}$

Εξοδος: Ελάχιστο πλήθος διαγραφών, για μετάβαση από την s στην t με μικρότερο ή ίσο από B κόστος.

1. While $L \leq R$:
 - 2 $mid \leftarrow \text{floor}((L + R)/2)$
 - 3 If Decision_Problem(G, s, t, B, mid) = NAI :
 - 4 $result \leftarrow mid$
 - 5 $R \leftarrow mid - 1$
 - 6 Else:
 - 7 $L \leftarrow mid + 1$
- 8 Return $result$

Αλγόριθμος 2: Decision_Problem($G = (V, E, l), s, t, B, k$)

Είσοδος: Κατευθυνόμενο γράφημα $G = (V, E, l)$ με συνάρτηση βαρών $l : E \rightarrow \mathbb{N}$, $s, t \in V(G)$, μέγιστο κόστος $B \in \mathbb{N}$ και πλήθος διαγραφών βαρέων ακμών

Εξοδος: ΝΑΙ (ΑΛΗΘΗΣ), ΟΧΙ (ΨΕΥΔΗΣ)

1. Αρχικοποιούμε ένα πίνακα $D[i, j]$, ο οποίος αντιστοιχεί στην ελάχιστη απόσταση της s από την i μετά τον μηδενισμό j ακμών
2. (For) $\forall v \in V$:
 3. (For) $\forall j \in \{0, \dots, k\}$:
 4. $D[v, j] \leftarrow \infty$
 5. $D[s, 0] \leftarrow 0$
 6. $Q \leftarrow \emptyset$ Min Priority Queue with key: cost $(_, c, _)$
 - % Για κάθε κορυφή v , θεωρούμε την εξής τριάδα: (v, c, z) όπου το c αντιπροσωπεύει το κόστος για να μεταβούμε σε αυτήν και το z το πλήθος των μηδενισμών που έχει εκτελέσει για να φτάσουμε σε αυτήν
 7. $Q.enqueue((s, 0, 0))$
 8. While $Q \neq \emptyset$
 9. $(u, c, z) \leftarrow Q.dequeue()$
 10. If $(u = t) \wedge (c \leq B)$:
 11. Return ΝΑΙ
 12. If $c > D[u, z]$:
 13. Προχώρησε % Αν έχουμε βρεί καλύτερο τρόπο να φτάσουμε στην u με z μηδενισμούς
 14. (For) $\forall (u, v) \in E$:
 - Επιλογή 1: Μη μηδενισμός ακμής
 15. $c' \leftarrow c + l(u, v)$
 16. If $(c' \leq B) \wedge (c' < D[v, z])$:
 17. $D[v, z] \leftarrow c'$
 18. $Q.enqueue((v, c', z))$
 - Επιλογή 2: Μηδενισμός ακμής (εφ'όσον είναι εφικτό)
 19. If $z < k$:
 20. $c' \leftarrow c$ % το κόστος να μεταβούμε θεωρείται 0
 21. If $(c' \leq B) \wedge (c' < D[v, z + 1])$:
 22. $D[v, z + 1] \leftarrow c'$
 23. $Q.enqueue((v, c', z + 1))$
 - Αν δεν φτάσαμε την t (εκτελώντας το μέγιστο δυνατό πλήθος μηδενισμών) με κόστος $c \leq B$
 24. Return ΟΧΙ (FALSE)

Σημειώσεις:

1. Θα μπορούσαμε να βελτιώσουμε το Optimization_Problem τρέχοντας ένας BFS ώστε να βρούμε την ελάχιστη έστω $d \in \mathbb{N}$ απόσταση ακμών του s από την t . Έτσι θα ξεκινούσαμε την δυαδική αναζήτηση με άνω όριο το d (σε ανάλυση χειρότερης περίπτωσης $d = |E|$, αλλά επιφέρει πρακτική βελτίωση)
2. Ισοδύναμα το decision problem θα μπορούσε να λυθεί ως εξής: θεωρούμε το επαυξημένο γράφημα G' όπου για κάθε κορυφή τοποθετούμε k καινούργιες ($v \rightarrow v_i, i \in \{0, 1, \dots, k\}$). Επίσης κατασκευάζουμε ακμές ώστε για κάθε ακμή (u, v) τοποθετούμε ακμές μεταξύ κάθε u, v "ίδιου επιπέδου" δηλαδή (u_i, v_i) με κόστος $l(u, v)$ και ακμές μεταξύ $(u_i, v_{i+1}), i \in \{0, 1, \dots, k-1\}$ με κόστος 0, δηλαδή $l(u_i, v_{i+1}) = 0$.
Τώρα τρέχουμε Dijkstra στο επαυξημένο γράφημα G' με σκοπό να φτάσουμε σε κάποια $t_i, i \in [k]$ με κόστος $\leq B$.